

RAPPORT DU PROJET II

1. Introduction

Ce mini-projet consiste à développer une application Java permettant de simuler une partie du fonctionnement d'un réseau GSM. Le système représente les communications entre les utilisateurs mobiles appelés MS (Mobile Station) et les stations de base BTS (Base Transceiver Station).

Dans un réseau GSM réel, les utilisateurs mobiles communiquent avec le réseau à travers des cellules radio contrôlées par des BTS. Chaque BTS possède une capacité limitée et gère plusieurs utilisateurs mobiles connectés dans sa zone de couverture.

L'objectif principal de ce projet est d'appliquer les concepts de la programmation orientée objet (POO) dans un contexte de télécommunications. Le projet permet également de comprendre le rôle des stations BTS, la gestion des utilisateurs mobiles et les échanges d'appels dans un réseau GSM simplifié.

Ce travail met en pratique plusieurs notions importantes du langage Java telles que :

- les classes et objets,
- l'encapsulation,
- les constructeurs,
- les interfaces,
- les collections,
- les exceptions,
- ainsi que le polymorphisme.

2. Objectifs

- Comprendre le fonctionnement d'un réseau GSM
- Utiliser la programmation orientée objet (classes, objets, interfaces)
- Modéliser un réseau mobile en Java
- Gérer les utilisateurs mobiles (MS)
- Simuler les appels entre utilisateurs
- Gérer les stations BTS et leur capacité
- Mettre en pratique les collections dynamiques
- Implémenter une gestion simple des exceptions
- Comprendre les interactions entre les différents composants du réseau GSM

3. Architecture du projet

Le projet est organisé en plusieurs packages afin de respecter une structure claire et professionnelle.

Le système contient principalement deux packages :

- **Package model**

Ce package contient les classes principales du système GSM :

- MS : représente un utilisateur mobile du réseau
- BTS : représente une station de base
- Reseau : représente le réseau GSM global
- Appelable : interface utilisée pour les communications

- **Package test**

Ce package contient la classe :

- TestReseau : utilisée pour tester toutes les fonctionnalités du projet

Cette organisation facilite :

- la maintenance du projet,
- la réutilisation des classes,
- et la compréhension globale du système.

4. Fonctionnement du programme

Le programme commence par la création d'un réseau GSM appelé **4G Campus**.

Ensuite, une ou plusieurs stations BTS sont ajoutées au réseau avec leurs caractéristiques :

- numéro,
- emplacement,
- rayon de couverture,
- puissance d'émission,
- capacité maximale d'utilisateurs.

Après la création des BTS, plusieurs utilisateurs mobiles MS sont créés puis attachés à une station de base.

Chaque utilisateur mobile possède :

- un nom,
- un prénom,
- un numéro de téléphone.

Le système permet ensuite aux utilisateurs de :

- s'appeler entre eux,
- recevoir des appels,
- enregistrer les appels reçus dans un historique.

Le programme affiche également :

- les informations du réseau,
- les BTS disponibles,
- les utilisateurs connectés,
- les appels réalisés,
- ainsi que les historiques des communications.

Le système intègre également une gestion des contraintes réseau :

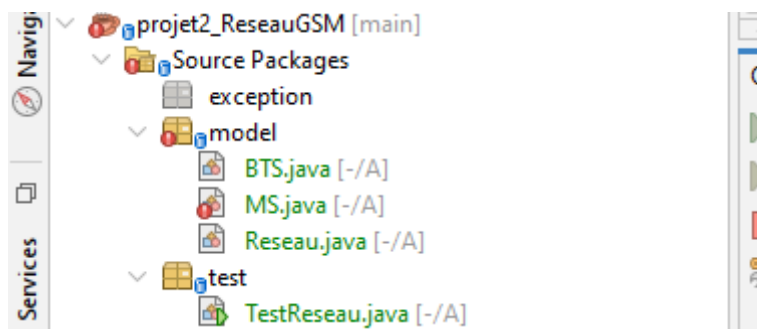
- limitation de capacité des BTS,
- prévention de surcharge réseau,
- gestion des exceptions en cas de saturation.

Cela permet de reproduire un environnement simplifié proche des réseaux GSM réels.

5. Étapes de réalisation et tests

1. Création du projet NetBeans

La première étape a consisté à créer le projet Java dans NetBeans ainsi que les différents packages nécessaires.



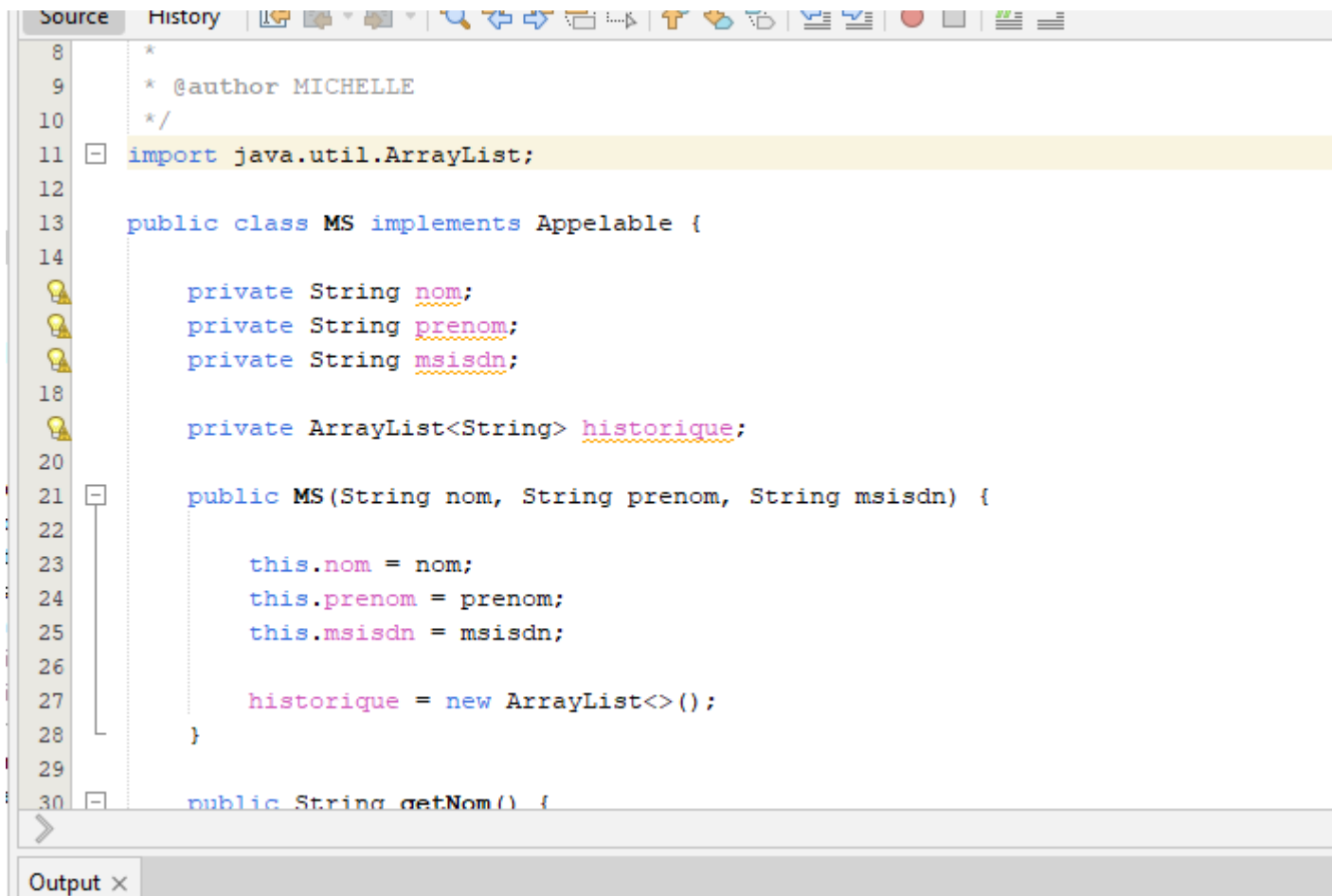
Cette figure montre la capture du projet NetBeans

2. Développement des classes

Les différentes classes du système GSM ont ensuite été développées :

- MS

- BTS
- Reseau
- Appellable
- TestReseau



The screenshot shows an IDE window with a source code editor. The code is in French and defines a class `MS` that implements the `Appellable` interface. The code includes an import for `java.util.ArrayList`, private fields for `nom`, `prenom`, `msisdn`, and `historique` (an `ArrayList<String>`). It also shows the constructor `MS` and the `getNom` method. The IDE has a toolbar at the top and a sidebar on the left with line numbers and foldable regions. The bottom of the window shows an 'Output' tab.

```
8      *
9      * @author MICHELLE
10     */
11     import java.util.ArrayList;
12
13     public class MS implements Appellable {
14
15         private String nom;
16         private String prenom;
17         private String msisdn;
18
19         private ArrayList<String> historique;
20
21         public MS(String nom, String prenom, String msisdn) {
22
23             this.nom = nom;
24             this.prenom = prenom;
25             this.msisdn = msisdn;
26
27             historique = new ArrayList<>();
28         }
29
30         public String getNom() {
```

```

29
30 public String getNom() {
31     return nom;
32 }
33
34 public String getPrenom() {
35     return prenom;
36 }
37
38 public String getMsisdn() {
39     return msisdn;
40 }
41
42 @Override
43 public void appeler(MS destinataire) {
44
45     String message =
46         nom + " appelle "
47         + destinataire.getNom()
48         + " (" + destinataire.getMsisdn() + ")";
49
50     System.out.println(message);
51

```

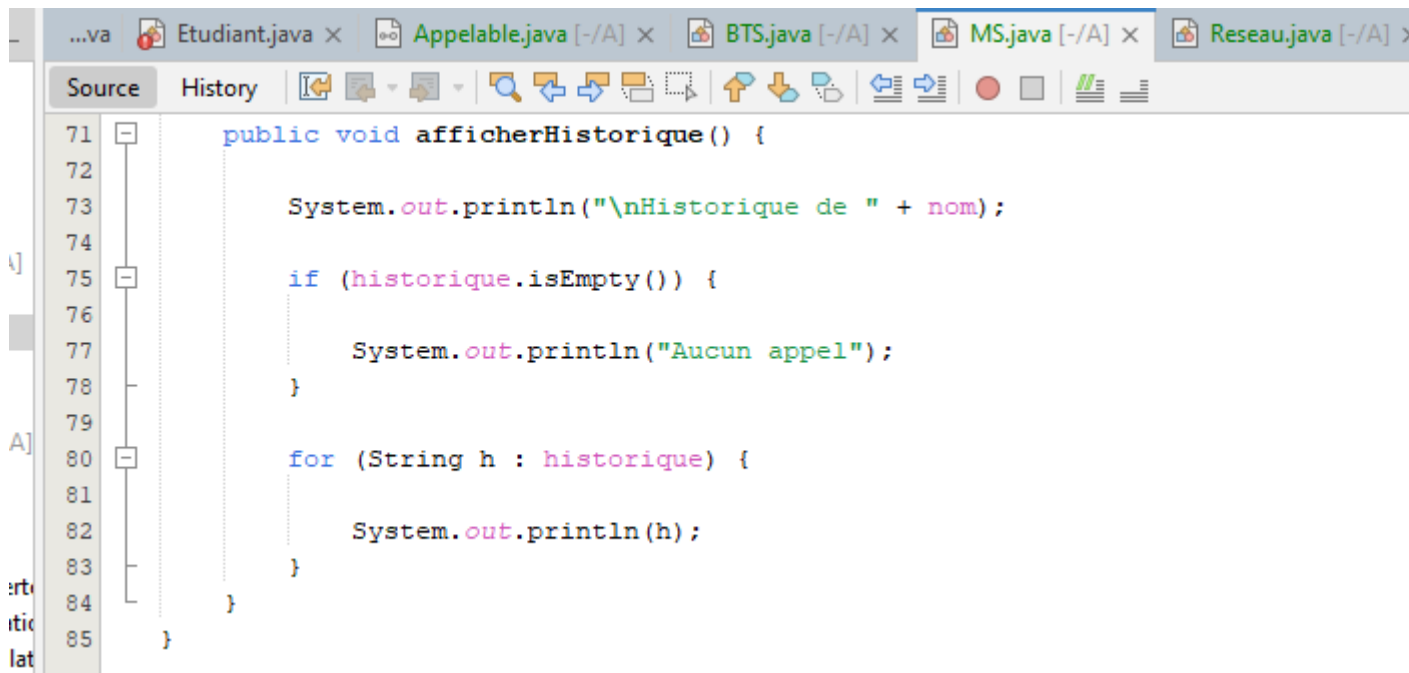
...va Etudiant.java x Appelleable.java [-/A] x BTS.java [-/A] x MS.java [-/A] x Reseau.java [-/A] x

Source History

```

52     destinataire.recevoirAppel(message);
53 }
54
55 @Override
56 public void recevoirAppel(String message) {
57
58     historique.add(message);
59 }
60
61 public void afficher() {
62
63     System.out.println(
64         nom + " "
65         + prenom
66         + " | "
67         + msisdn
68     );
69 }
70
71 public void afficherHistorique() {
72
73     System.out.println("\nHistorique de " + nom);
74

```



```
71 public void afficherHistorique() {
72
73     System.out.println("\nHistorique de " + nom);
74
75     if (historique.isEmpty()) {
76
77         System.out.println("Aucun appel");
78     }
79
80     for (String h : historique) {
81
82         System.out.println(h);
83     }
84 }
85 }
```

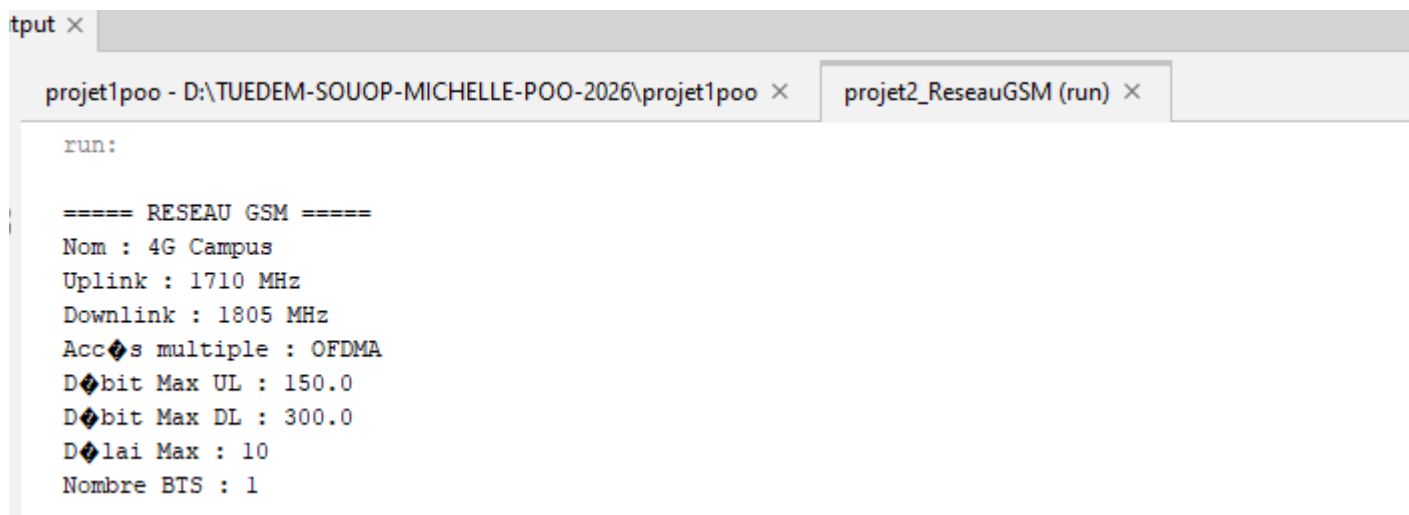
Cette capture montre le code source

3. Exécution du programme

Le programme a ensuite été exécuté afin de vérifier le bon fonctionnement du réseau GSM simulé.

Le système affiche :

- les informations du réseau,
- les BTS disponibles,
- les utilisateurs connectés.



```
run:

===== RESEAU GSM =====
Nom : 4G Campus
Uplink : 1710 MHz
Downlink : 1805 MHz
Accès multiple : OFDMA
Débit Max UL : 150.0
Débit Max DL : 300.0
Délai Max : 10
Nombre BTS : 1
```

```

===== BTS =====
Numero : 1
Emplacement : Douala-Akwa
Hauteur : 30.0
Milieu : Urbain
Rayon : 2.5
Puissance : 40.0
Utilisateurs connectés : 3
Michelle Tuedem | 111
Dr Yankam | 222
Tuedem Michelle | 333

===== APPELS =====
Michelle appelle Dr (222)
Dr appelle Tuedem (333)
Tuedem appelle Michelle (111)

Historique de Michelle
Tuedem appelle Michelle (111)

Historique de Dr
Michelle appelle Dr (222)

Historique de Tuedem
Dr appelle Tuedem (333)

```

Cette capture nous montre l'exécution du programme

4. Simulation des appels

Plusieurs appels ont été simulés entre les utilisateurs mobiles :

- Michelle appelle Dr Yankam
- Dr Yankam appelle Tuedem
- Tuedem appelle Michelle

```

===== APPELS =====
Michelle appelle Dr (222)
Dr appelle Tuedem (333)
Tuedem appelle Michelle (111)

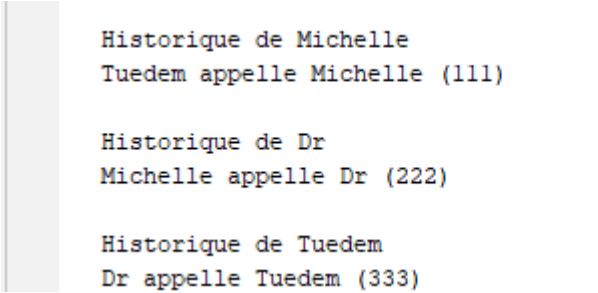
```

5. Affichage des historiques des appels

Chaque utilisateur mobile affiche ensuite l'historique des appels reçus.

Cette fonctionnalité permet de vérifier :

- la transmission correcte des appels,
- l'enregistrement des communications,
- et le bon fonctionnement global du système.



```
Historique de Michelle  
Tuedem appelle Michelle (111)  
  
Historique de Dr  
Michelle appelle Dr (222)  
  
Historique de Tuedem  
Dr appelle Tuedem (333)
```

Cette capture montre les historiques d'appels

6. Conclusion

Ce mini-projet a permis de comprendre le fonctionnement simplifié d'un réseau GSM ainsi que les interactions entre les utilisateurs mobiles et les stations BTS.

Le projet a également permis de mettre en pratique plusieurs notions importantes de la programmation orientée objet en Java :

- classes,
- objets,
- encapsulation,
- interfaces,
- polymorphisme,
- collections,
- et gestion des exceptions.

Grâce à cette simulation, il a été possible de représenter les communications mobiles dans un environnement proche des réseaux télécoms réels.

Ce travail constitue une bonne introduction à la modélisation des systèmes réseaux en programmation orientée objet et pourrait être amélioré dans une future version avec :

- plusieurs BTS connectées entre elles,
- la gestion des SMS,
- la mobilité des utilisateurs,
- ou encore une interface graphique.